НИУ ВШЭ · Факультет компьютерных наук

Департамент программной инженерии

Индивидуальное задание № 3

Автогонки на кольцевой клеточной трассе

Разработка имитационной программы с использованием взаимодействующих процессов

Исполнитель	Учебные сведения
Сайгитов Уллубий Русланович	Группа БПИ 258
Вариант № 20 — «Автогонки»	Дисциплина: «Архитектура компьютера и операционные системы»
Язык C, ОС Linux	Дата отчёта: 8 ноября 2026 года

Исходный код и реализации задания
https://github.com/Ullubiy07/auto-racing

Содержание работы

Рассмотрены версии с дочерними процессами и двумя видами POSIX-семафоров, а также текущая версия с независимо запускаемыми race и car. Описаны архитектура, интерфейс, завершение, фактическое тестирование и использование ИИ.

Основание: «Требования к выполнению индивидуального задания № 3», 2026. Номера уровней оценки далее обозначают требуемую архитектуру, а не гарантию итоговой оценки.

Москва · 2026

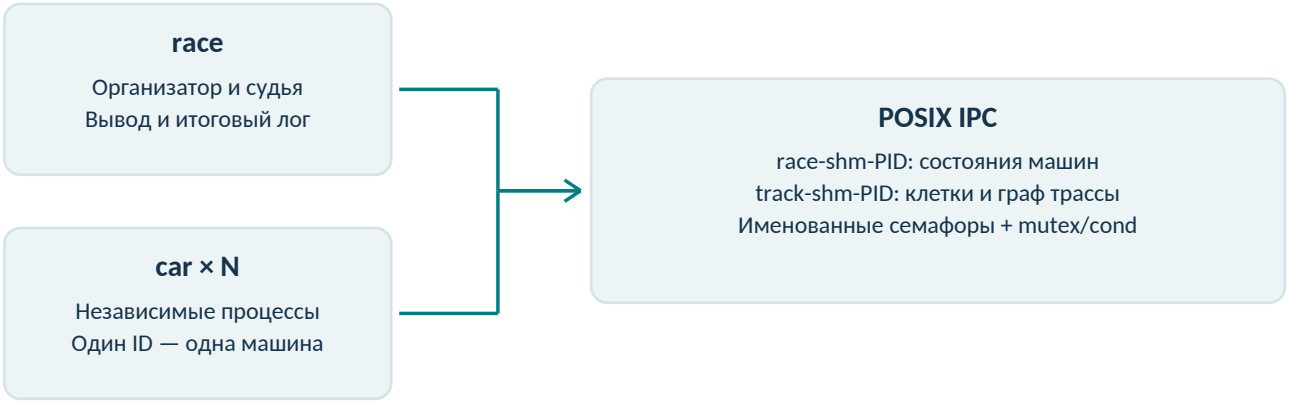
1. Задача и варианты реализации

Предметный сценарий

На кольцевой клеточной трассе соревнуются машины. Машина занимает одну клетку, выбирает допустимое направление, пытается занять соседнюю клетку и освобождает прежнюю. Нельзя занять клетку, в которой уже находится другая машина. Переход через стартовую линию используется для подсчёта кругов.

Машины чередуют выбор хода по расстоянию до финиша и случайный выбор. Пауза между ходами случайно выбирается из заданного диапазона. Организатор отображает трассу и таблицу лидеров. Гонка завершается при достижении лимита кругов, сходе всех машин либо отмене пользователем.

Сущности и процессы



В независимой реализации судья — модуль judge.c внутри процесса race. Он не запускается как отдельный процесс. Каждая участвующая машина представлена отдельным процессом car. Команда хранит название и цвет; самостоятельного поведения и отдельного процесса у команды нет.

Реализации в репозитории

Уровень ТЗ	Ветка	Организация
4–5	task-3/multi-process	Родитель создаёт машины через fork; неименованные sem_t находятся в POSIX shared memory.
6	named-semaphores	Родитель создаёт машины через fork; синхронизация клеток и машин — именованные POSIX-семафоры.
7–8	independent-processes	Две программы race и car запускаются независимо; общая память POSIX и именованные семафоры.
Выше 8	Не реализовано	Вариант с каналами или POSIX-очередями сообщений в данную работу не входит.

Текущий отчёт подробно описывает independent-processes. Обе прежние версии отдельно собраны из временных копий веток и проверены в двух сценариях: финиш двух машин на data/circle и SIGINT во время стартового отсчёта. Во всех четырёх запусках получены код 0 и ожидаемый статус в логе.

Предыдущая версия	Финиш	SIGINT до старта
task-3/multi-process	Пройден	Пройден
named-semaphores	Пройден	Пройден

2. Данные и синхронизация

Размещение общей памяти

```text
race-shm-PID  : [RaceResources][Car 0][Car 1] ... [Car N-1]
track-shm-PID : [Track][Cell* для строк организатора][массив Cell]
```

Организатор создаёт объекты через shm_open, задаёт размер ftruncate и отображает их mmap с MAP_SHARED. Машина открывает объекты по PID организатора, получает их размеры через fstat и строит локальные указатели от собственного адреса отображения.

В общих Car хранятся ID, массив имени водителя, команда, координаты, круги и флаги. Связи Road представлены координатами Point, а Entity ссылается на машину через ID. Это позволяет независимо запущенным процессам работать при разных базовых адресах mmap.

Track.map и таблица указателей на строки в текущем формате используются организатором. Машина не разыменовывает эти чужие адреса: она вычисляет начало массива Cell по известной разметке и заполняет собственную map. Указатели от sem_open хранятся локально в Race и CarProcess.

Назначение синхропримитивов

Объект / начальное значение	Назначение
cell-PID-Y-X / 1	Взаимоисключение при чтении и изменении занятости конкретной клетки.
race-PID-car-ID / 1	Защита изменяемого состояния машины; используется и машиной, и судьёй.
race-cars-ready-PID / 0	Одна машина сообщает о готовности; организатор ожидает заданное число уведомлений.
race-cars-stopped-PID / 0	Подтверждение завершения работы машины с общими ресурсами.
RaceState.mutex и cond	PTHREAD_PROCESS_SHARED; общий старт, остановка и прерываемые по условию задержки.
Атомарные флаги	isStarted, isOver, isInterrupted и isOut. Атомарность отдельного флага не заменяет защиту составных операций.

Именованные семафоры создаёт race с O_CREAT | O_EXCL. Процессы car открывают те же имена без O_CREAT. При EINTR вызов sem_wait повторяется. При завершении каждый процесс закрывает свои ссылки через sem_close; организатор удаляет имена через sem_unlink.

Независимые процессы не являются дочерними для race, поэтому ожидание их штатного завершения реализовано подтверждениями, а не waitpid. В версиях с fork родитель использует ожидание детей.

3. Выполнение и завершение

Последовательность запуска

1. race разбирает параметры, создаёт трассу, общие данные и именованные семафоры. После инициализации устанавливается обработчик SIGINT.

2. Каждый car получает ID и PID организатора из CLI, подключает память, резервирует ID под общим mutex и открывает семафоры. Повторно занять уже зарегистрированный ID нельзя. При ошибке подключения своё резервирование отменяется.

3. Машина заполняет общие сведения о водителе и команде, отправляет уведомление готовности и ждёт старта. Организатор ожидает нужное число машин, размещает активные машины на решётке и инициализирует судью.

4. После стартовых огней race выставляет isStarted и будит машины. Процессы выбирают ходы, обновляют положение и круги. Родитель периодически отображает карту и таблицу; координаты машины читаются под её семафором.

Обычный финиш

При достижении лимита кругов устанавливается isOver, ожидающие процессы пробуждаются. Организатор ждёт carsStopped от всех машин, учтённых в carsReadyCount, в том числе выбывших. Каждая машина отправляет уведомление после закрытия своих локальных ресурсов. Затем сохраняется лог и удаляются ресурсы организатора.

SIGINT и сход машины

SIGINT организатора меняет isInterrupted. Обработчик установлен через sigaction без SA_RESTART; логика остановки выполняется вне обработчика. SIGINT машины меняет атомарный isOut; цикл движения прекращается, занятая клетка освобождается под соответствующими семафорами.

Ожидание старта в car использует pthread_cond_timedwait и проверяет отмену примерно каждые 100 мс. Время задержек вычисляется по CLOCK_MONOTONIC. Сигнал не гарантирует мгновенного выхода из уже выполняемого timedwait; при большой настроенной задержке реакция может быть отложена до её окончания.

Перед munmap обработчик SIGINT отключается. При сходе всех машин гонка заканчивается без победителя: статус All cars are out, поле Winner — None. Обработчики не вызывают sem_wait и не выполняют освобождение ресурсов.

Откат частичной инициализации

Ошибка возвращается вызывающей функции. Метки cleanup выполняют очистку уже созданных объектов в обратном порядке. Для таблицы семафоров клеток учитывается количество успешных открытий; неинициализированные элементы не закрываются.

Ограничения штатной схемы: ещё не согласованы одновременное новое подключение и начало остановки; не реализовано восстановление после аварийной смерти машины. Подробности и границы проверки приведены в разделе 7.

4. Сборка и запуск по веткам

Требуются Linux, GCC с C23 и CMake ≥ 4.4.3. Команды выполняются из корня репозитория. Перед переключением сохраните изменения и завершите гонку. Все ветки используют bin: после переключения пересоберите выбранную версию с --clean-first.

4.1. На 4–5 баллов: неименованные семафоры

```bash
git switch task-3/multi-process
cmake -S . -B build-unnamed -DCMAKE_BUILD_TYPE=Debug
cmake --build build-unnamed --clean-first
mkdir -p logs
cd bin
# По умолчанию: 14 машин и карта twisty
./race
# Запуск с параметрами на короткой карте
./race --team "Red Bull:cyan:Max,Liam" --start "Max,Liam" \
  --map ../data/circle --laps 1 --draw-delay 20 \
  --car-delay 20:40 --output ../logs/unnamed.log
cd ..
```

Без параметров: 14 машин и twisty. С параметрами: две машины, circle, лог logs/unnamed.log. race сам создаёт дочерние процессы. Для проверки Ctrl+C используйте --laps 1000.

4.2. На 6 баллов: именованные семафоры

```bash
git switch named-semaphores
cmake -S . -B build-named -DCMAKE_BUILD_TYPE=Debug
cmake --build build-named --clean-first
mkdir -p logs
cd bin
# По умолчанию: 14 машин и карта twisty
./race
# Запуск с параметрами на короткой карте
./race --team "Red Bull:cyan:Max,Liam" --start "Max,Liam" \
  --map ../data/circle --laps 1 --draw-delay 20 \
  --car-delay 20:40 --output ../logs/named.log
cd ..
```

Без параметров: 14 машин и twisty. С параметрами: две машины, circle, лог logs/named.log. race создаёт дочерние процессы с именованными семафорами. Проверка Ctrl+C — аналогично версии 4.1.

4.3. На 7–8 баллов: независимые процессы

Терминал 1. После сборки выберите один из двух запусков: со стандартной картой twisty или с параметрами на circle. Опция --cars обязательна в обоих случаях; race ждёт две машины.

```bash
git switch independent-processes
cmake -S . -B build-independent -DCMAKE_BUILD_TYPE=Debug
cmake --build build-independent --clean-first
mkdir -p logs
cd bin
# Настройки по умолчанию, обязательное число машин
./race --cars 2
# Альтернативный запуск с параметрами
./race --cars 2 --map ../data/circle --laps 1 \
  --car-delay 20:40 --output ../logs/independent.log
```

Для каждого нового запуска race подключите машины командами со следующей страницы, используя PID именно этой гонки.

4.3. Подключение независимых машин

Терминал 2. Откройте корень того же репозитория. Команды ниже предполагают одну проверяемую гонку: pgrep выбирает PID последнего запущенного race. При нескольких гонках используйте pgrep -a -x race и задайте RACE_PID вручную.

```bash
cd bin
RACE_PID=$(pgrep -n -x race)
./car --id 0 --driver Max --team "Red Bull" \
  --color 36 --race-pid "$RACE_PID" &
./car --id 1 --driver Liam --team "Red Bull" \
  --color 36 --race-pid "$RACE_PID" &
wait
```

После подключения двух машин начинаются стартовые огни и гонка. В стандартном запуске лог получает имя с датой и временем в logs; в запуске с параметрами — logs/independent.log. В отличие от веток 4.1 и 4.2, здесь race не создаёт процессы машин: каждая команда car запускает самостоятельную программу.

Для проверки прерывания всей гонки нажмите Ctrl+C в терминале 1. Для проверки схода отдельной машины запустите её без фонового & в отдельном терминале и нажмите Ctrl+C там. Если выбыли все машины, ожидается All cars are out и Winner: None.

4.4. Опции race в independent-processes

Опция	Назначение
-c, --cars N	Обязательное число машин: 1–26.
-l, --laps N	Лимит кругов; по умолчанию 1.
-d, --draw-delay MS	Интервал отрисовки; по умолчанию 50 мс.
--car-delay MIN:MAX	Диапазон задержек между ходами; по умолчанию 80:180 мс.
-m, --map FILE	Файл трассы; по умолчанию ../data/twisty.
--map-only	Отрисовка без таблицы лидеров.
-o, --output FILE	Путь к текстовому логу; иначе имя с датой и временем в ../logs.
-h, --help	Справка. Опции --team и --start есть только в предыдущих двух версиях.

Опции car

Все пять параметров обязательны: -i/--id, -n/--driver, -t/--team, -c/--color, -r/--race-pid. ID начинается с нуля и определяет стартовый порядок; цвет — ANSI-код 31–37. Имя водителя и команды: не более 12 байт каждое. Полная справка доступна через ./car --help и ./race --help.

Файловые пути самого приложения разрешаются относительно текущей рабочей директории. Поэтому примеры запускаются из bin и создают logs заранее. Скрипты run_tests.sh и start_cars.sh далее относятся только к ветке independent-processes; старые версии проверяются командами из разделов 4.1 и 4.2.

5. Автоматизированные проверки

run_tests.sh адаптирован к независимому запуску процессов. Старые тесты --team/--start заменены запуском race и отдельных car. Используются карта data/circle, таймауты, контроль кодов возврата, содержимого отчёта, stderr и удаления тестовых объектов в /dev/shm.

Итог фактического прогона 08.11.2026: **26 проверок пройдено, 0 ошибок.** Проверялись свежие сборки текущих исходников с -std=gnu2x -Wall -Wextra -Werror -pthread. Это подтверждает перечисленные сценарии, но не доказывает отсутствие всех возможных ошибок.

№	Сценарий	Результат
1	Справка race	Пройден
2	Справка car	Пройден
3	Не указано число машин	Пройден
4	Нулевое число машин	Пройден
5	Превышен лимит машин	Пройден
6	Отрицательное число кругов	Пройден
7	Нулевая задержка отрисовки	Пройден
8	Обратный диапазон задержек машин	Пройден
9	Отсутствующий файл карты	Пройден
10	Не указаны обязательные опции car	Пройден
11	Отрицательный ID	Пройден
12	Недопустимый ANSI-цвет	Пройден
13	Неположительный PID гонки	Пройден
14	Слишком длинное имя водителя	Пройден
15	Слишком длинное имя команды	Пройден
16	Финиш двух независимых машин	Пройден
17	Четыре машины, три круга, задержка 0:0	Пройден
18	Отрисовка только карты	Пройден
19	SIGINT без подключённых машин	Пройден
20	SIGINT с одной подключённой машиной	Пройден
21	SIGINT во время стартового отсчёта	Пройден
22	SIGINT во время гонки	Пройден
23	SIGINT одной машины: сход	Пройден
24	Все машины сошли, победителя нет	Пройден
25	Сход машины до подключения второй	Пройден
26	Повторный ID отклонён	Пройден

Артефакты этого прогона: bin/test_logs/run.RQdPR0/test-1 … test-26. Для каждого теста сохранены вывод проверок; для интеграционных — stdout/stderr процессов и report.log. Полный прогон не выполнялся заново на исторических ветках.

6. Результаты и скрипты запуска

Пример: штатный финиш двух машин

```text
Status              : Completed successfully
Winner              : TestDriver0
Duration            : 0.10 s
Participants        : 2
```

Победитель TestDriver0 прошёл один круг. Длительность отражает интервал после стартовых огней; отсчёт в неё не включён. Конкретный победитель и длительность при других запусках могут отличаться.

Пример: все машины сошли

```text
Status              : All cars are out
Winner              : None
Duration            : 0.07 s
Participants        : 2
```

Запуск тестов и готового состава

```bash
# Из корня проекта
./run_tests.sh

# 14 машин из семи команд, из bin
./race --cars 14
# В другом терминале: PID заменить на фактический
./start_cars.sh 28576
```

run_tests.sh определяет корень проекта по собственному расположению и использует абсолютные пути к bin/race, bin/car и data/circle. Поэтому запуск тестов не привязан к текущей директории bin. Стандартный таймаут одного ожидания — 20 секунд; TEST_TIMEOUT позволяет его изменить.

start_cars.sh запускает ID 0–13 параллельно и ждёт их завершения: Red Bull — Max, Liam (36); Ferrari — Charles, Lewis (31); Mercedes — George, Kimi (34); McLaren — Lando, Oscar (33); Aston Martin — Fernando, Lance (32); Alpine — Pierre, Esteban (35); Williams — Alex, Carlos (37). Проверены аргументы всех 14 запусков с подставным исполняемым файлом. Полная гонка из 14 машин в описанный прогон из 26 тестов не входит.

7. Критическая оценка и ограничения

Подключение во время остановки. carsReadyCount отражает принятые уведомления. Уже подключающаяся машина может ещё не попасть в этот счётчик, когда race начинает удалять mutex/cond. Для устранения требуется согласованный протокол регистрации и закрытия подключения; joinLock пока не реализован.

Аварийная смерть процесса. После SIGKILL или падения машина не отправит carsStopped; организатор может ждать бесконечно. Если процесс погиб с захваченным семафором, может блокироваться и судья. Проверки аварийного убийства не входят в заявленные успешные тесты.

Входные данные. По выбранному упрощению числа разбираются atoi; строгий контроль синтаксиса и переполнения не реализован. Полная валидация произвольных повреждённых карт также не заявляется. Эти ограничения не следует считать исправленными по результату штатных тестов.

8. Использование ИИ

ИИ (ChatGPT / Codex) использовался как вспомогательный инструмент. Он создал start_cars.sh, обновил run_tests.sh и подготовил данный PDF-отчёт. Дополнительно ИИ добавил CLI в car.c по образцу cli.c. В переписке также разбирались POSIX IPC, сигналы, очистка ресурсов и ошибки реализации; выдавались примеры кода, которые автор адаптировал.

Основные запросы к ИИ

Запросы изложены в отредактированном виде с сохранением смысла; они не являются дословными цитатами переписки.

Скрипт запуска машин. Создать shell-скрипт start_cars.sh для независимого запуска 14 машин семи заданных команд. Сохранить имена водителей и цвета команд, передавать PID организатора аргументом скрипта.

Автоматизированное тестирование. Обновить run_tests.sh с учётом независимого запуска процессов, нового интерфейса командной строки и изменений в обработке завершения гонки.

Интерфейс программы car. Добавить в car.c разбор коротких и длинных опций, справку и проверки аргументов, сохранив стиль реализации cli.c.

Подготовка отчёта. Подготовить PDF-отчёт по требованиям индивидуального задания № 3. Описать реализацию и результаты проверки, указать участие ИИ в создании тестов, отчёта и скрипта start_cars.sh.

Проверка и исправление предложений ИИ

Предложения не принимались без проверки: код перечитывался, собирался с предупреждениями как ошибками, выполнялись сценарии CLI и интеграционные тесты. При переходе от fork к независимым процессам общие указатели были заменены координатами/ID и локальными отображениями; исправлены чтение координат до захвата семафора, очистка чужим семафором и доступ к неинициализированному судье.

В тестовом скрипте исключён eval, добавлены независимый запуск машин, таймауты и сохранение диагностического вывода. Отчёт не повторяет прежние утверждения о гарантированной защите от всех гонок и о мгновенной реакции на любой сигнал.

Основания отчёта: локальное ТЗ i-tasks-req-03.pdf (разделы требований и отчёта); исходники src/; ветки task-3/multi-process, named-semaphores, independent-processes; текущие скрипты запуска; фактические логи run.RQdPR0. Сведения о группе и варианте перенесены из прежнего отчёта исполнителя по ИЗ №2.